PROJECT REPORT

On

“Recruitment & HR Analytics Portal”

[**Experiential Learning**]

Submitted By:

Gunjan R. Chimote[CM25078]

Guided By:

Ms.Suchita Mesakar



DEPARTMENT OF CSE (AI &ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR.

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2026-2027

© S.B.J.I.T.M.R Nagpur

INDEX

INDEX		i
CHAPTER 1	INTRODUCTION	1
CHAPTER 2	METHODOLOGY	2
CHAPTER 3	TOOLS/PLATFORMS	3-4
CHAPTER 4	DESIGN & IMPLEMENTATION	5-24
	4.1 ALGORITHM	
	4.2 FLOWCHART	
	4.3 SOURCE CODE	
CHAPTER 5	RESULT & DISCUSSION	25-27
	5.1 OUTPUT	
	5.2 DISCUSSION	
	5.3 APPLICATION	
CHAPTER 6	CONCLUSION	28
	REFERENCES	29

CHAPTER 1

INTRODUCTION

Recruitment is an important process in Human Resource Management (HRM) that helps organisations identify and select suitable candidates for job vacancies. Managing candidate information manually can be time-consuming and may lead to difficulties in searching records, tracking application status, and analysing recruitment data. HR Analytics helps organisations understand recruitment information and make better decisions using data. The Recruitment & HR Analytics Portal is a Python-based project designed to simplify basic recruitment activities. It allows users to add candidate details, view candidate records, search for candidates, update application status, and analyse recruitment data. The project uses Python programming and CSV file handling to store and manage information efficiently.

Objectives

The main objective of this project is to develop a simple and user-friendly recruitment management system using Python. The project aims to maintain candidate records in a structured format, simplify searching and updating candidate information, track recruitment stages, and generate basic HR analytics. It also provides a simple skill-matching feature that compares candidate skills with the skills required for a particular job role.

The portal provides several features, including adding new candidates, displaying all candidate records, searching candidates by name, ID, role, or skill, and updating application status. It also displays the total number of candidates, average work experience, applications by job role, and candidate counts by recruitment status. A skill-matching feature provides a percentage based on the overlap between required skills and candidate skills.

This project is developed as an educational prototype for understanding Python programming, file handling, and basic data analytics. It can be used to demonstrate recruitment record management and basic reporting. The current version uses a local CSV file and a command-line interface. It does not include advanced security, online access, or automated hiring decisions.

CHAPTER 2

METHODOLOGY

The development of the Recruitment & HR Analytics Portal follows a systematic approach. First, the basic requirements of a recruitment management system are identified. The information required for each candidate, such as candidate ID, name, email, applied role, skills, experience, application status, and application date, is determined.

Next, the data structure is created using a CSV file named candidates.csv. Python functions are developed to perform individual operations such as adding candidates, displaying records, searching information, updating application status, and generating analytics. Input validation is included to reduce common data-entry errors and prevent duplicate email addresses.

The analytics module processes the stored candidate information and calculates basic statistics. The skill-matching module compares the skills provided by a candidate with the skills required for a job role. Finally, the different features are tested using sample candidate records to check whether the program performs the intended operations.

When the program starts, it checks whether the candidate data file exists. If the file is unavailable, the program creates it with sample records. The main menu is then displayed, allowing the user to select an operation.

Based on the selected option, the program performs the required task. Candidate information is read from the CSV file, processed using Python, and displayed to the user. When a record is added or updated, the changes are saved to the file. The menu continues to appear until the user selects the exit option.

The system can be tested by viewing existing candidates, adding a new candidate, searching for a particular candidate, updating application status, and displaying the analytics dashboard. Additional tests include entering invalid experience values, attempting to add a duplicate email address, and selecting an invalid menu option. These tests help verify the basic functionality and input validation of the application.

The final testing stage involves checking the exit option and confirming that saved changes remain available when the program is executed again. These tests are intended to verify the basic functions of the academic prototype. Actual results should be confirmed by running the program in the intended Python environment.

Since the project deals with candidate information, privacy and responsible data handling are important considerations. Fictional records are used for demonstration, and real personal information should not be entered into the sample application. The current implementation stores data in a local CSV file and does not provide user authentication, encryption, access control, or secure multi-user storage. Consequently, it is not suitable for managing confidential recruitment records in a real organisation without substantial improvements.

CHAPTER 3

TOOLS/PLATFORMS

1.1 SOFTWARE REQUIREMENT

The Recruitment & HR Analytics Portal is developed using Python and several supporting technologies to provide an interactive platform for recruitment management, candidate tracking, and HR data analysis. The following software requirements are necessary to develop and run the application.

1. Operating System

The application can run on Windows 10 or Windows 11. It can also be adapted for Linux and macOS systems that support Python and the required libraries.

2. Programming Language — Python

Python is the primary programming language used to develop the application. It handles the application's logic, candidate data processing, input validation, database operations, and analytical calculations. Its simple syntax and extensive library support make it suitable for developing this project.

3. Integrated Development Environment (IDE) — Visual Studio Code

Visual Studio Code (VS Code) is used to write, edit, debug, and execute the Python program. It provides syntax highlighting, error detection, terminal access, and extensions that simplify Python application development.

4. Streamlit

Streamlit is a Python framework used to create the web-based user interface. It allows users to interact with the portal through forms, buttons, navigation menus, tables, and other interface components without requiring separate HTML, CSS, or JavaScript development.

5. SQLite Database

SQLite is used to store and manage recruitment-related information in a local database. It supports structured data storage and retrieval, including candidate details, application statuses, job information, and other records implemented in the application.

6. Pandas Library

Pandas is used for data manipulation and analysis. It helps organize candidate records into tabular structures, filter information, calculate statistics, summarize recruitment data, and prepare records for CSV export.

7. Plotly Library

Plotly can be used to create interactive charts and graphical representations of recruitment data, such as candidate statistics, application status distributions, and recruitment trends. It is required if the project code uses Plotly for its visualizations.

8. Python Package Manager — pip

Pip is used to install and manage the Python libraries required by the application. It helps ensure that dependencies such as Streamlit, Pandas, and Plotly are available in the Python environment.

9. Web Browser

A modern web browser, such as Google Chrome, Microsoft Edge, or Mozilla Firefox, is required to access the Streamlit application. The portal can be opened locally using the URL provided by Streamlit, typically http://localhost:8501.

10. Python Virtual Environment

A virtual environment can be created to keep the project's dependencies separate from other Python

projects. This helps avoid package conflicts and makes the development environment easier to manage.

11. CSV File Support

CSV file support enables the application to import or export tabular recruitment information, depending on the implemented features. It is useful for sharing records and performing additional analysis using spreadsheet applications.

12. Required Python Packages

The main packages required for the project can be installed through the VS Code terminal:

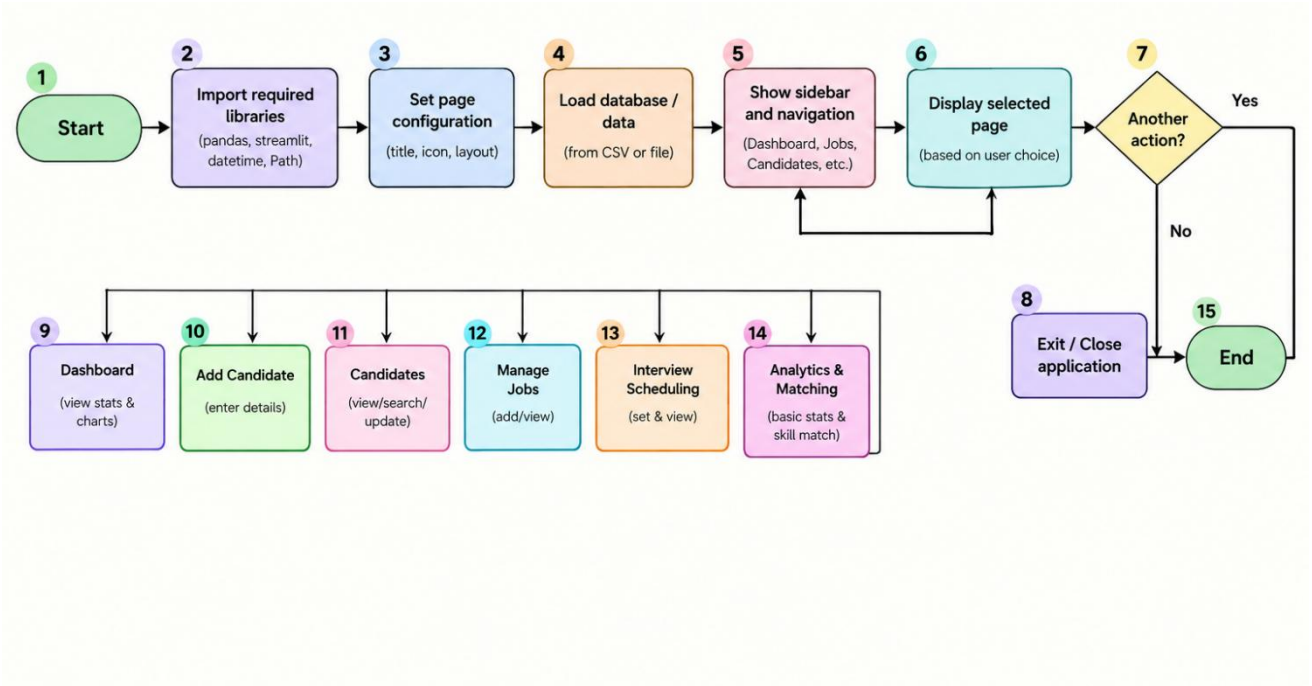
CHAPTER 4

DESIGN & IMPLEMENTATION

4.1 ALGORITHM

1. Start the program.
2. Check whether the candidate CSV file exists.
3. Create the file with sample records if it does not exist.
4. Display the main menu.
5. Accept the user's choice.
6. If the user selects Add Candidate, accept and validate the details and save the record.
7. If the user selects View Candidates, read and display all candidate records.
8. If the user selects Search Candidates, search the stored records using the entered keyword.
9. If the user selects Update Status, find the candidate and update the application status.
10. If the user selects HR Analytics, calculate and display candidate statistics.
11. If the user selects Skill-Match Suggestions, compare candidate skills with the required skills and display the matching percentage.
12. Repeat the menu until the user selects Exit.
13. Stop the program.

4.2 FLOWCHART



4.3 SOURCE CODE

```
 import os
import re
import sqlite3
from datetime import date, datetime
from io import BytesIO

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

DB_NAME = "recruitment_hr.db"
STATUSES = ["Applied", "Shortlisted", "Interview", "Selected", "Rejected"]

st.set_page_config(
    page_title="Recruitment & HR Analytics",
    page_icon="🚀",
    layout="wide"
)

st.markdown("""
<style>
.stApp {
    background: linear-gradient(135deg, #0f172a, #172554);
    color: #e2e8f0;
}
[data-testid="stSidebar"] {
    background: #020617;
}
[data-testid="stMetric"] {
    background: #1e293b;
    padding: 16px;
    border-radius: 12px;
    border: 1px solid #334155;
}
h1, h2, h3 { color: #bfdbfe; }
.stButton button {
    border-radius: 8px;
    font-weight: 600;
}
</style>
""", unsafe_allow_html=True)

def connect():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys=ON")
```

```
    return conn

def init_db():
    with connect() as con:
        con.executescript("""
        CREATE TABLE IF NOT EXISTS jobs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            department TEXT NOT NULL,
            skills TEXT DEFAULT '',
            experience TEXT DEFAULT '',
            vacancies INTEGER NOT NULL DEFAULT 1,
            status TEXT NOT NULL DEFAULT 'Open',
            created_date TEXT NOT NULL
        );

        CREATE TABLE IF NOT EXISTS candidates (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE,
            phone TEXT DEFAULT '',
            qualification TEXT DEFAULT '',
            skills TEXT DEFAULT '',
            experience TEXT DEFAULT '',
            created_date TEXT NOT NULL
        );

        CREATE TABLE IF NOT EXISTS applications (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            candidate_id INTEGER NOT NULL,
            job_id INTEGER NOT NULL,
            application_date TEXT NOT NULL,
            status TEXT NOT NULL DEFAULT 'Applied',
            UNIQUE(candidate_id, job_id),
            FOREIGN KEY(candidate_id) REFERENCES candidates(id)
                ON DELETE CASCADE,
            FOREIGN KEY(job_id) REFERENCES jobs(id)
                ON DELETE CASCADE
        );

        CREATE TABLE IF NOT EXISTS interviews (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            application_id INTEGER NOT NULL,
            interview_date TEXT NOT NULL,
            interview_time TEXT NOT NULL,
            interviewer TEXT NOT NULL,
            result TEXT NOT NULL DEFAULT 'Pending',
            feedback TEXT DEFAULT '',
            FOREIGN KEY(application_id) REFERENCES applications(id)
                ON DELETE CASCADE
        );
```

```
        """)

        # Add columns when upgrading an older version of the database.
        for table, column, definition in [
            ("interviews", "feedback", "TEXT DEFAULT ""),
        ]:
            existing = [
                row["name"]
                for row in con.execute(
                    f"PRAGMA table_info({table})"
                ).fetchall()
            ]
            if column not in existing:
                con.execute(
                    f"ALTER TABLE {table} ADD COLUMN {column} {definition}"
                )

def read_df(query, params=()):
    with connect() as con:
        return pd.read_sql_query(query, con, params=params)

def run_sql(query, params=()):
    with connect() as con:
        cur = con.execute(query, params)
        return cur.lastrowid

def rows(query, params=()):
    with connect() as con:
        return [
            dict(r) for r in
            con.execute(query, params).fetchall()
        ]

def valid_email(email):
    return re.fullmatch(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        email.strip()
    ) is not None

def skill_set(value):
    return {
        x.strip().lower()
        for x in re.split(r"[,;\n]", value or "")
        if x.strip()
    }

def match_percentage(candidate_skills, required_skills):
    required = skill_set(required_skills)
    candidate = skill_set(candidate_skills)

    if not required:
```

```
        return 0, []

    matched = sorted(required & candidate)
    score = round(100 * len(matched) / len(required), 1)
    return score, matched

def load_application_data():
    return read_df("""
        SELECT a.id AS application_id,
               c.id AS candidate_id,
               c.name AS candidate,
               c.email,
               c.skills AS candidate_skills,
               c.qualification,
               c.experience AS candidate_experience,
               j.id AS job_id,
               j.title AS job,
               j.department,
               j.skills AS required_skills,
               j.vacancies,
               a.application_date,
               a.status
        FROM applications a
        JOIN candidates c ON c.id = a.candidate_id
        JOIN jobs j ON j.id = a.job_id
        ORDER BY a.id DESC
    """)
def header():
    st.markdown("""
    <div style="
        padding:24px;border-radius:16px;
        background:linear-gradient(90deg,#1d4ed8,#2563eb);
        margin-bottom:20px">
        <h1 style="color:white;margin:0">
            🚀 Recruitment & HR Analytics Portal
        </h1>
        <p style="color:#dbeafe;margin-bottom:0">
            Recruitment management • Candidate tracking • Data-driven insights
        </p>
    </div>
    """, unsafe_allow_html=True)

def section_title(title, subtitle=None):
    st.subheader(title)
    if subtitle:
        st.caption(subtitle)

def safe_delete(table, record_id):
    if table not in {"jobs", "candidates"}:
        return
    run_sql(f"DELETE FROM {table} WHERE id=?", (record_id,))
```

```
def dashboard():
    section_title(" 📊 Executive Dashboard")

    jobs = read_df("SELECT * FROM jobs")
    candidates = read_df("SELECT * FROM candidates")
    apps = load_application_data()
    interviews = read_df("SELECT * FROM interviews")

    total_apps = len(apps)
    selected = (
        int((apps["status"] == "Selected").sum())
        if not apps.empty else 0
    )
    selection_rate = (
        round(selected / total_apps * 100, 1)
        if total_apps else 0
    )

    metrics = st.columns(5)
    metrics[0].metric("Total Jobs", len(jobs))
    metrics[1].metric(
        "Open Jobs",
        int((jobs["status"] == "Open").sum())
        if not jobs.empty else 0
    )
    metrics[2].metric("Candidates", len(candidates))
    metrics[3].metric("Applications", total_apps)
    metrics[4].metric("Selection Rate", f"{selection_rate}%")

    left, right = st.columns(2)

    with left:
        section_title("Recruitment Pipeline")
        if not apps.empty:
            pipeline = (
                apps["status"].value_counts()
                .reindex(STATUSES, fill_value=0)
                .rename_axis("Status")
                .reset_index(name="Candidates")
            )
            fig = px.funnel(
                pipeline,
                x="Candidates",
                y="Status",
                color="Status",
                title="Applications by Status"
            )
            fig.update_layout(
                template="plotly_dark",
                paper_bgcolor="rgba(0,0,0,0)",
```

```
            plot_bgcolor="rgba(0,0,0,0)"
        )
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.info("Add applications to see the recruitment funnel.")

with right:
    section_title("Applications by Department")
    if not apps.empty:
        dept = (
            apps["department"].value_counts()
            .rename_axis("Department")
            .reset_index(name="Applications")
        )
        fig = px.pie(
            dept, names="Department", values="Applications",
            hole=0.45, title="Department Distribution"
        )
        fig.update_layout(
            template="plotly_dark",
            paper_bgcolor="rgba(0,0,0,0)"
        )
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.info("Department analytics will appear here.")

section_title("🗓️ Monthly Application Trend")
if not apps.empty:
    trend = apps.copy()
    trend["application_date"] = pd.to_datetime(
        trend["application_date"], errors="coerce"
    )
    trend = trend.dropna(subset=["application_date"])
    if not trend.empty:
        trend["Month"] = (
            trend["application_date"].dt.to_period("M").astype(str)
        )
        monthly = (
            trend.groupby("Month").size()
            .reset_index(name="Applications")
        )
        fig = px.line(
            monthly, x="Month", y="Applications",
            markers=True, title="Applications Over Time"
        )
        fig.update_layout(
            template="plotly_dark",
            paper_bgcolor="rgba(0,0,0,0)"
        )
        st.plotly_chart(fig, use_container_width=True)
```

```
    upcoming = read_df("""
        SELECT i.interview_date, i.interview_time,
               i.interviewer, c.name AS candidate, j.title AS job
        FROM interviews i
        JOIN applications a ON a.id=i.application_id
        JOIN candidates c ON c.id=a.candidate_id
        JOIN jobs j ON j.id=a.job_id
        WHERE i.interview_date >= ?
        ORDER BY i.interview_date, i.interview_time
        LIMIT 5
    """, (str(date.today()),))

    section_title("🗓️ Upcoming Interviews")
    if upcoming.empty:
        st.info("No upcoming interviews.")
    else:
        st.dataframe(upcoming, use_container_width=True, hide_index=True)

def jobs_page():
    section_title("💼 Job Management")

    with st.expander("➕ Create a Job Vacancy", expanded=False):
        with st.form("create_job"):
            title = st.text_input("Job Title")
            dept = st.text_input("Department")
            skills = st.text_input("Required Skills (comma-separated)")
            exp = st.text_input("Required Experience")
            vacancies = st.number_input("Vacancies", min_value=1, value=1)
            submitted = st.form_submit_button("Create Job")

            if submitted:
                if not title.strip() or not dept.strip():
                    st.error("Job title and department are required.")
                else:
                    run_sql("""
                        INSERT INTO jobs
                        (title, department, skills, experience,
                         vacancies, status, created_date)
                        VALUES (?, ?, ?, ?, ?, 'Open', ?)
                    """, (
                        title.strip(), dept.strip(), skills.strip(),
                        exp.strip(), int(vacancies), str(date.today())
                    ))
                    st.success("Job created.")
                    st.rerun()

    df = read_df("SELECT * FROM jobs ORDER BY id DESC")
    section_title("Job Vacancies")

    if df.empty:
```

```
    st.info("No jobs created yet.")
    return

search = st.text_input("Search jobs by title or department")
if search:
    df = df[
        df["title"].str.contains(search, case=False, na=False)
        | df["department"].str.contains(search, case=False, na=False)
    ]

st.dataframe(df, use_container_width=True, hide_index=True)

ids = df["id"].tolist()
if not ids:
    return

selected_id = st.selectbox(
    "Select a job to edit or delete",
    ids,
    format_func=lambda x: (
        f"#{x} — {df.loc[df.id == x, 'title'].iloc[0]}"
    )
)
record = rows("SELECT * FROM jobs WHERE id=?", (selected_id,))[0]

with st.form("edit_job"):
    new_title = st.text_input("Title", record["title"])
    new_dept = st.text_input("Department", record["department"])
    new_skills = st.text_input("Skills", record["skills"] or "")
    new_exp = st.text_input("Experience", record["experience"] or "")
    new_vacancies = st.number_input(
        "Vacancies", min_value=1, value=max(1, record["vacancies"])
    )
    new_status = st.selectbox(
        "Status", ["Open", "Closed"],
        index=0 if record["status"] == "Open" else 1
    )
    save = st.form_submit_button("Save Job Changes")

    if save:
        if not new_title.strip() or not new_dept.strip():
            st.error("Title and department are required.")
        else:
            run_sql("""
                UPDATE jobs SET title=?, department=?, skills=?,
                experience=?, vacancies=?, status=? WHERE id=?
            """, (
                new_title.strip(), new_dept.strip(), new_skills.strip(),
                new_exp.strip(), int(new_vacancies), new_status, selected_id
            ))
            st.success("Job updated.")
```

```
            st.rerun()

    if st.button("🗑️ Delete Selected Job"):
        safe_delete("jobs", selected_id)
        st.success("Job and its linked records were deleted.")
        st.rerun()

def candidates_page():
    section_title("👥 Candidate Management")

    with st.expander("➕ Register a Candidate"):
        with st.form("create_candidate"):
            name = st.text_input("Full Name")
            email = st.text_input("Email")
            phone = st.text_input("Phone")
            qualification = st.text_input("Qualification")
            skills = st.text_input("Skills (comma-separated)")
            experience = st.text_input("Experience")
            submitted = st.form_submit_button("Add Candidate")

            if submitted:
                if not name.strip() or not email.strip():
                    st.error("Name and email are required.")
                elif not valid_email(email.strip()):
                    st.error("Enter a valid email address.")
                else:
                    try:
                        run_sql("""
                            INSERT INTO candidates
                            (name, email, phone, qualification, skills,
                             experience, created_date)
                            VALUES (?, ?, ?, ?, ?, ?, ?)
                        """, (
                            name.strip(), email.strip().lower(),
                            phone.strip(), qualification.strip(), skills.strip(),
                            experience.strip(), str(date.today())
                        ))
                        st.success("Candidate registered.")
                        st.rerun()
                    except sqlite3.IntegrityError:
                        st.error("A candidate with this email already exists.")

    df = read_df("SELECT * FROM candidates ORDER BY id DESC")
    search = st.text_input("Search by name, email, or skills")

    if search and not df.empty:
        mask = (
            df["name"].str.contains(search, case=False, na=False)
            | df["email"].str.contains(search, case=False, na=False)
            | df["skills"].str.contains(search, case=False, na=False)
```

```
    )
    df = df[mask]

section_title("Candidate Database")
if df.empty:
    st.info("No matching candidates found.")
    return

st.dataframe(df, use_container_width=True, hide_index=True)

selected_id = st.selectbox(
    "Select candidate to edit",
    df["id"].tolist(),
    format_func=lambda x: (
        f"#{x} — {df.loc[df.id == x, 'name'].iloc[0]}"
    )
)
record = rows(
    "SELECT * FROM candidates WHERE id=?", (selected_id,)
)[0]

with st.form("edit_candidate"):
    new_name = st.text_input("Name", record["name"])
    new_email = st.text_input("Email", record["email"])
    new_phone = st.text_input("Phone", record["phone"] or "")
    new_qualification = st.text_input(
        "Qualification", record["qualification"] or ""
    )
    new_skills = st.text_input("Skills", record["skills"] or "")
    new_experience = st.text_input(
        "Experience", record["experience"] or ""
    )
    save = st.form_submit_button("Save Candidate Changes")

    if save:
        if not new_name.strip() or not valid_email(new_email.strip()):
            st.error("Provide a name and valid email.")
        else:
            try:
                run_sql("""
                    UPDATE candidates SET name=?, email=?, phone=?,
                    qualification=?, skills=?, experience=? WHERE id=?
                """, (
                    new_name.strip(), new_email.strip().lower(),
                    new_phone.strip(), new_qualification.strip(),
                    new_skills.strip(), new_experience.strip(), selected_id
                ))
                st.success("Candidate updated.")
                st.rerun()
            except sqlite3.IntegrityError:
                st.error("That email is already registered.")
```

```
    if st.button("🗑️ Delete Selected Candidate"):
        safe_delete("candidates", selected_id)
        st.success("Candidate and linked records deleted.")
        st.rerun()

def applications_page():
    section_title("📝 Application Tracking")

    candidates = rows("SELECT id, name FROM candidates ORDER BY name")
    jobs = rows(
        "SELECT id, title FROM jobs WHERE status='Open' ORDER BY title"
    )

    if candidates and jobs:
        candidate_options = {
            f"{c['name']} (ID {c['id']})": c["id"] for c in candidates
        }
        job_options = {
            f"{j['title']} (ID {j['id']})": j["id"] for j in jobs
        }

        with st.form("new_application"):
            candidate_label = st.selectbox(
                "Candidate", list(candidate_options.keys())
            )
            job_label = st.selectbox("Job", list(job_options.keys()))
            submitted = st.form_submit_button("Submit Application")

            if submitted:
                try:
                    run_sql("""
                        INSERT INTO applications
                        (candidate_id, job_id, application_date, status)
                        VALUES (?, ?, ?, 'Applied')
                    """, (
                        candidate_options[candidate_label],
                        job_options[job_label],
                        str(date.today())
                    ))
                    st.success("Application submitted.")
                    st.rerun()
                except sqlite3.IntegrityError:
                    st.error("This candidate already applied for this job.")
    else:
        st.info("Create at least one candidate and one open job first.")

    df = load_application_data()
    if df.empty:
        st.info("No applications yet.")
```

```
    return

col1, col2 = st.columns(2)
with col1:
    status_filter = st.multiselect(
        "Filter by status", STATUSES, default=STATUSES
    )
with col2:
    dept_filter = st.multiselect(
        "Filter by department",
        sorted(df["department"].dropna().unique().tolist())
    )

filtered = df[df["status"].isin(status_filter)]
if dept_filter:
    filtered = filtered[filtered["department"].isin(dept_filter)]

st.dataframe(
    filtered[[
        "application_id", "candidate", "job", "department",
        "application_date", "status"
    ]],
    use_container_width=True,
    hide_index=True
)

app_id = st.selectbox(
    "Select application to update",
    df["application_id"].tolist()
)
current = rows(
    "SELECT status FROM applications WHERE id=?", (app_id,)
)[0]["status"]

with st.form("update_application"):
    new_status = st.selectbox(
        "New Status", STATUSES, index=STATUSES.index(current)
    )
    update = st.form_submit_button("Update Application Status")

    if update:
        run_sql(
            "UPDATE applications SET status=? WHERE id=?",
            (new_status, app_id)
        )
        st.success("Application status updated.")
        st.rerun()

if st.button(" 🗑 Delete Selected Application"):
    run_sql("DELETE FROM applications WHERE id=?", (app_id,))
    st.success("Application deleted.")
```

```
        st.rerun()

def interviews_page():
    section_title(" Interview Scheduling & Evaluation")

    eligible = rows("""
        SELECT a.id, c.name, j.title
        FROM applications a
        JOIN candidates c ON c.id=a.candidate_id
        JOIN jobs j ON j.id=a.job_id
        WHERE a.status IN ('Applied', 'Shortlisted', 'Interview')
        ORDER BY c.name
    """)

    if eligible:
        options = {
            f"Application {r['id']}: {r['name']} — {r['title']}": r["id"]
            for r in eligible
        }

        with st.form("schedule_interview"):
            app_label = st.selectbox("Application", list(options.keys()))
            interview_date = st.date_input(
                "Interview Date", value=date.today(), min_value=date.today()
            )
            interview_time = st.time_input("Interview Time")
            interviewer = st.text_input("Interviewer Name")
            submitted = st.form_submit_button("Schedule Interview")

            if submitted:
                if not interviewer.strip():
                    st.error("Enter the interviewer's name.")
                else:
                    app_id = options[app_label]
                    run_sql("""
                        INSERT INTO interviews
                        (application_id, interview_date, interview_time,
                         interviewer, result, feedback)
                        VALUES (?, ?, ?, ?, 'Pending', '')
                    """, (
                        app_id, str(interview_date),
                        interview_time.strftime("%H:%M"),
                        interviewer.strip()
                    ))
                    run_sql(
                        "UPDATE applications SET status='Interview' WHERE id=?",
                        (app_id,)
                    )
                    st.success("Interview scheduled.")
                    st.rerun()
    else:
```

```
    st.info("No eligible applications. Add an application first.")

df = read_df("""
    SELECT i.id AS interview_id, i.application_id,
           c.name AS candidate, j.title AS job,
           i.interview_date, i.interview_time,
           i.interviewer, i.result, i.feedback
    FROM interviews i
    JOIN applications a ON a.id=i.application_id
    JOIN candidates c ON c.id=a.candidate_id
    JOIN jobs j ON j.id=a.job_id
    ORDER BY i.interview_date DESC, i.interview_time DESC
""")

section_title("Interview Records")
if df.empty:
    st.info("No interviews scheduled.")
    return

st.dataframe(df, use_container_width=True, hide_index=True)

interview_id = st.selectbox(
    "Select interview to evaluate", df["interview_id"].tolist()
)
record = rows(
    "SELECT * FROM interviews WHERE id=?", (interview_id,)
)[0]

with st.form("evaluate_interview"):
    result = st.selectbox(
        "Interview Result",
        ["Pending", "Selected", "Rejected", "On Hold"],
        index=["Pending", "Selected", "Rejected", "On Hold"].index(
            record["result"]
            if record["result"] in
            ["Pending", "Selected", "Rejected", "On Hold"]
            else "Pending"
        )
    )
    feedback = st.text_area(
        "Interview Feedback", record["feedback"] or ""
    )
    submitted = st.form_submit_button("Save Evaluation")

    if submitted:
        run_sql("""
            UPDATE interviews SET result=?, feedback=? WHERE id=?
        """, (result, feedback.strip(), interview_id))

        if result in ("Selected", "Rejected"):
            run_sql("""
```

```
            UPDATE applications SET status=? WHERE id=?
        """, (result, record["application_id"]))

        st.success("Interview evaluation saved.")
        st.rerun()


def analytics_page():
    section_title("📈 HR Analytics & Candidate Matching")

    df = load_application_data()
    if df.empty:
        st.info("Add applications to generate recruitment analytics.")
    else:
        total = len(df)
        selected = int((df["status"] == "Selected").sum())
        rejected = int((df["status"] == "Rejected").sum())
        selection_rate = round(selected / total * 100, 1)

        a, b, c, d = st.columns(4)
        a.metric("Applications", total)
        b.metric("Selected", selected)
        c.metric("Rejected", rejected)
        d.metric("Selection Rate", f"{selection_rate}%")

        department = (
            df.groupby(["department", "status"]).size()
            .reset_index(name="Count")
        )
        fig = px.bar(
            department, x="department", y="Count", color="status",
            barmode="group", title="Recruitment Status by Department"
        )
        fig.update_layout(template="plotly_dark")
        st.plotly_chart(fig, use_container_width=True)

        top_jobs = (
            df.groupby("job").size()
            .reset_index(name="Applications")
            .sort_values("Applications", ascending=False)
            .head(10)
        )
        fig = px.bar(
            top_jobs, x="Applications", y="job", orientation="h",
            title="Top Jobs by Application Volume"
        )
        fig.update_layout(template="plotly_dark", yaxis_autorange="reversed")
        st.plotly_chart(fig, use_container_width=True)

        st.download_button(
```

```
    " 📥 Download Filterable Application Data (CSV)",
    data=df.to_csv(index=False).encode("utf-8"),
    file_name="recruitment_report.csv",
    mime="text/csv"
)

if st.button("Prepare PDF Summary"):
    try:
        from reportlab.lib import colors
        from reportlab.lib.pagesizes import A4
        from reportlab.platypus import (
            SimpleDocTemplate, Paragraph, Spacer, Table,
            TableStyle
        )
        from reportlab.lib.styles import getSampleStyleSheet

        buffer = BytesIO()
        doc = SimpleDocTemplate(buffer, pagesize=A4)
        styles = getSampleStyleSheet()
        story = [
            Paragraph("Recruitment & HR Analytics Report",
                      styles["Title"]),
            Spacer(1, 16),
            Paragraph(f"Generated: {datetime.now():%Y-%m-%d %H:%M}",
                      styles["Normal"]),
            Spacer(1, 12),
            Paragraph(f"Total Applications: {total}", styles["Normal"]),
            Paragraph(f"Selected Candidates: {selected}",
                      styles["Normal"]),
            Paragraph(f"Rejected Applications: {rejected}",
                      styles["Normal"]),
            Paragraph(f"Selection Rate: {selection_rate}%",
                      styles["Normal"]),
            Spacer(1, 18)
        ]

        summary = (
            df.groupby("status").size()
            .reset_index(name="Applications")
        )
        table_data = [summary.columns.tolist()] + summary.values.tolist()
        table = Table(table_data, repeatRows=1)
        table.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1D4ED8")),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
            ("GRID", (0, 0), (-1, -1), 0.5, colors.grey),
            ("PADDING", (0, 0), (-1, -1), 8)
        ]))
        story.append(table)
        doc.build(story)
```

```
            st.download_button(
                "⬇️ Download PDF Report",
                data=buffer.getvalue(),
                file_name="hr_analytics_report.pdf",
                mime="application/pdf"
            )
        except ImportError:
            st.error("Install reportlab using: pip install reportlab")

st.divider()
section_title(
    "🎯 Skills-Based Candidate Matching",
    "Matches are based on required skills only, not an ML prediction."
)

jobs = rows(
    "SELECT id, title, skills FROM jobs WHERE status='Open' ORDER BY title"
)
candidates = rows(
    "SELECT id, name, skills, qualification, experience FROM candidates ORDER BY name"
)

if not jobs or not candidates:
    st.info("Add at least one open job and one candidate to use matching.")
    return

job_id = st.selectbox(
    "Choose a job for matching",
    [j["id"] for j in jobs],
    format_func=lambda x: next(
        j["title"] for j in jobs if j["id"] == x
    )
)
job = next(j for j in jobs if j["id"] == job_id)
required = skill_set(job["skills"])

if not required:
    st.warning("Add required skills to this job to calculate matches.")
    return

match_results = []
for candidate in candidates:
    score, matched = match_percentage(
        candidate["skills"], job["skills"]
    )
    match_results.append({
        "Candidate": candidate["name"],
        "Qualification": candidate["qualification"],
        "Experience": candidate["experience"],
        "Match %": score,
```

```
            "Matched Skills": ", ".join(matched) or "None",
            "Missing Skills": ", ".join(
                sorted(required - skill_set(candidate["skills"]))
            ) or "None"
        })

    match_df = pd.DataFrame(match_results).sort_values(
        "Match %", ascending=False
    )
    st.dataframe(match_df, use_container_width=True, hide_index=True)

    fig = px.bar(
        match_df, x="Match %", y="Candidate", orientation="h",
        title=f"Skills Match — {job['title']}",
        range_x=[0, 100]
    )
    fig.update_layout(template="plotly_dark", yaxis_autorange="reversed")
    st.plotly_chart(fig, use_container_width=True)

    st.caption(
        "A skills match score is decision support only. "
        "HR should assess qualifications, experience, and interviews "
        "before making hiring decisions."
    )

def main():
    init_db()
    header()

    st.sidebar.title("🚀 Navigation")
    st.sidebar.caption("Recruitment & HR Analytics Portal")
    page = st.sidebar.radio(
        "Choose a module",
        [
            "Dashboard",
            "Jobs",
            "Candidates",
            "Applications",
            "Interviews",
            "Analytics & Matching"
        ]
    )

    st.sidebar.divider()
    st.sidebar.caption("Experiential Learning Project")
    st.sidebar.caption("Serial No.: 43")
    st.sidebar.caption("USN: CM25078")

    if page == "Dashboard":
        dashboard()
    elif page == "Jobs":
```

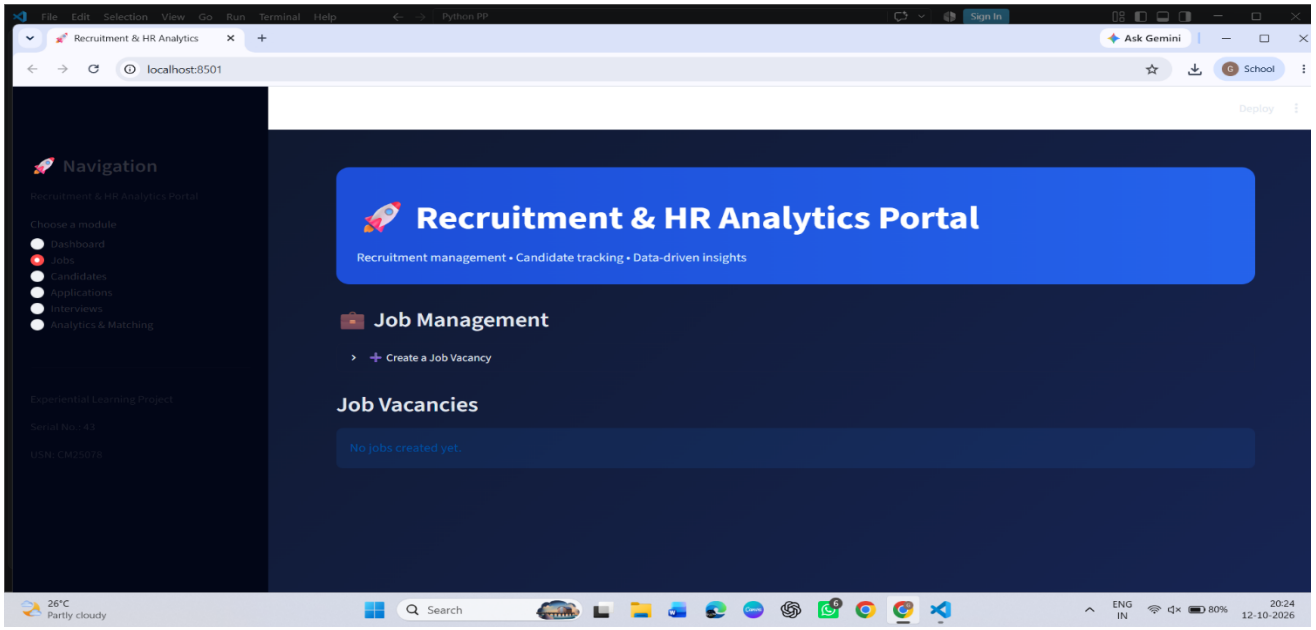
```
        jobs_page()
    elif page == "Candidates":
        candidates_page()
    elif page == "Applications":
        applications_page()
    elif page == "Interviews":
        interviews_page()
    elif page == "Analytics & Matching":
        analytics_page()

if __name__ == "__main__":
    main()
```

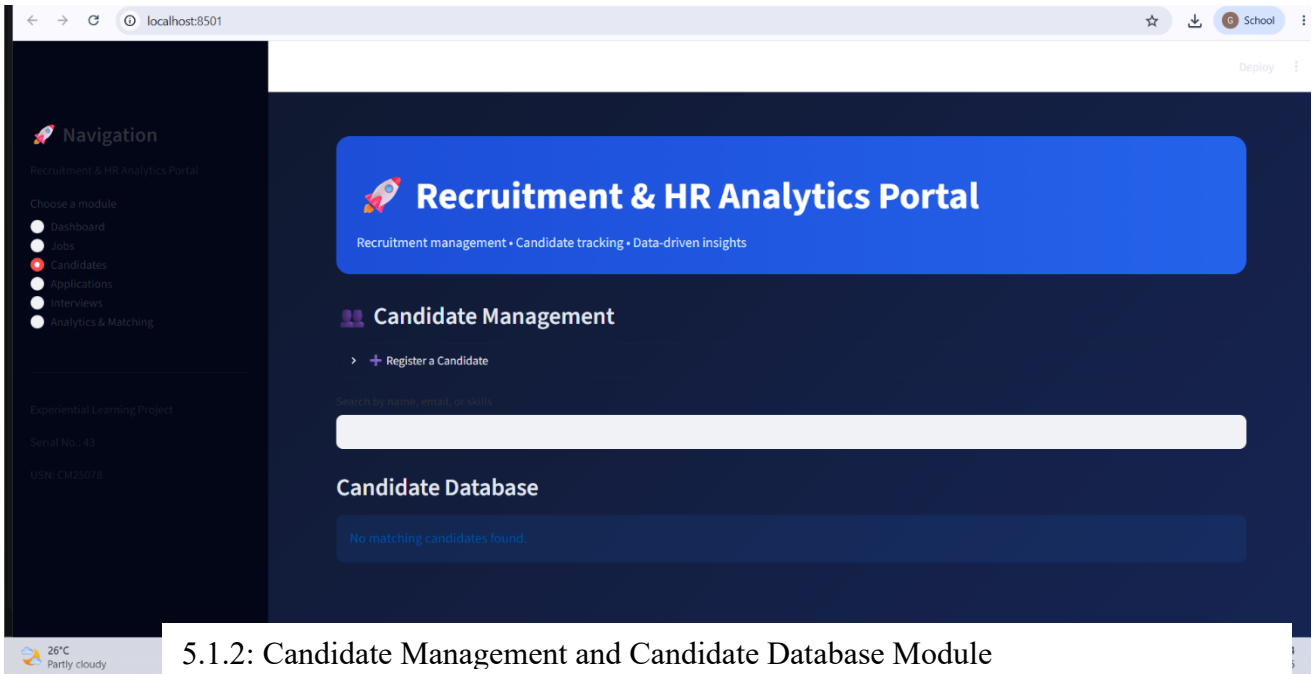
CHAPTER 5

RESULT & DISCUSSION

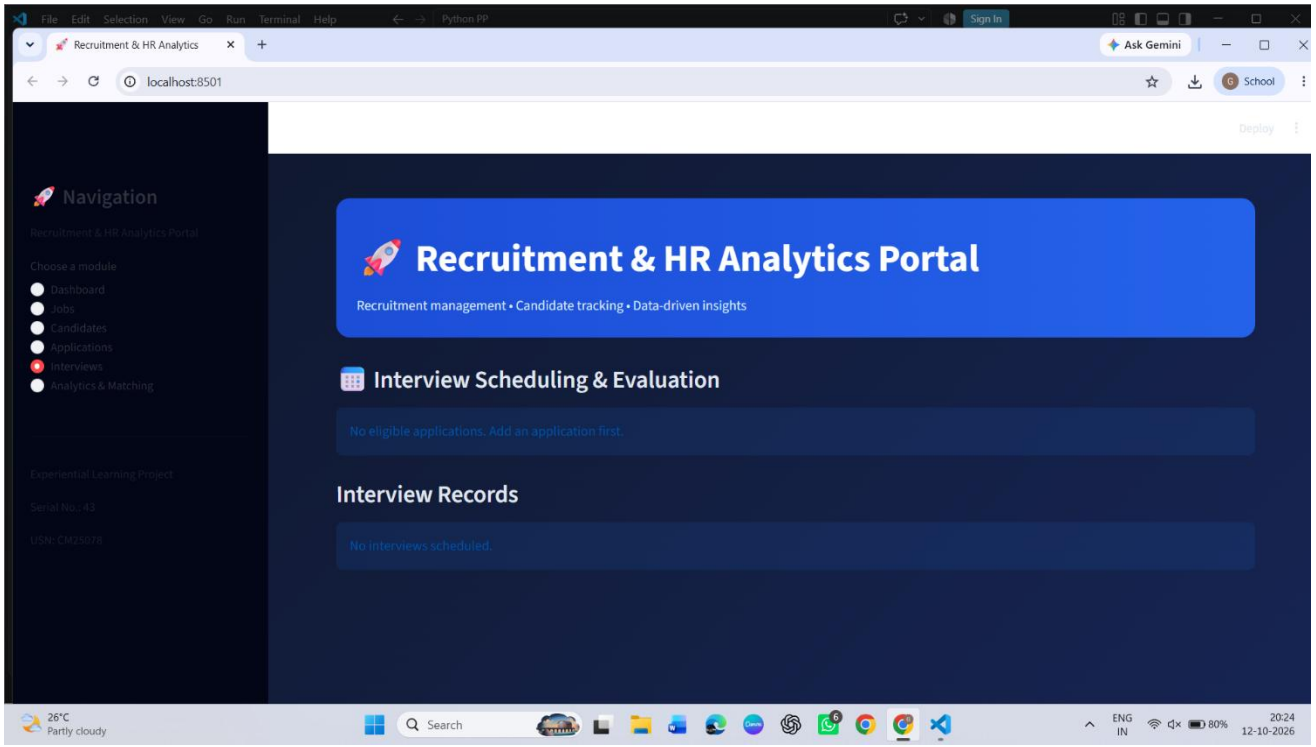
5.1 OUTPUT



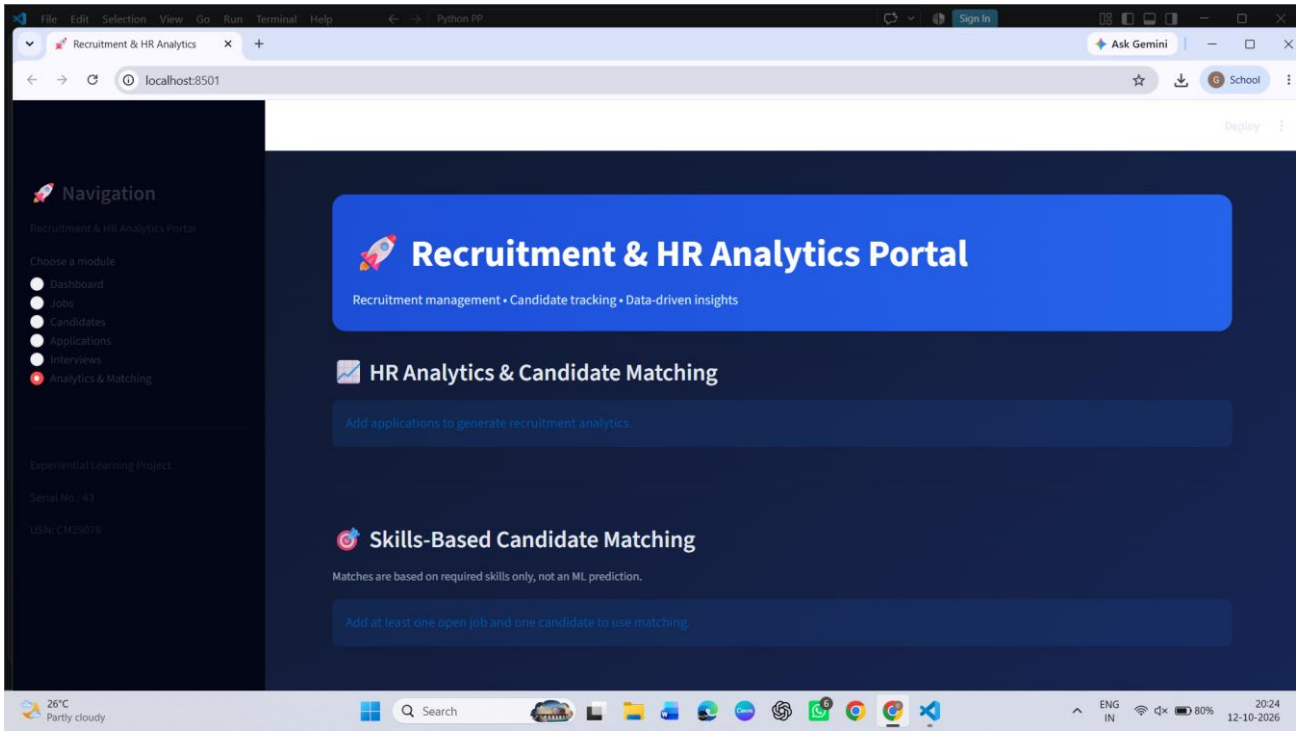
5.1.1: Home Page of the Recruitment & HR Analytics Portal



5.1.2: Candidate Management and Candidate Database Module



5.1.3 Interview Scheduling and Evaluation Module



5.1.4: HR Analytics Dashboard and Skills-Based Candidate Matching Module

5.2 DISCUSSION

The project demonstrates how Python can be used to organise candidate information and perform basic recruitment analytics. The use of functions makes the program modular, while CSV file handling allows information to be stored and retrieved between executions. Input validation helps reduce common data-entry errors, and the analytics dashboard presents recruitment information in an understandable format.The skill-matching feature provides a basic comparison between required skills and candidate skills. However, the percentage is only an indicator of skill overlap and does not measure a candidate's complete suitability for a job. Human review is necessary for recruitment decisions.The current implementation is suitable for academic demonstration and small sample datasets. It does not provide multi-user access, authentication, encryption, or advanced database management. These features could be considered in future versions.

5.3 APPLICATION

1. Candidate Management: To store, organize, and maintain candidate information.
2. Job Vacancy Management: To create and manage job vacancy records.
3. Application Tracking: To monitor candidate application status throughout the recruitment process.
4. Interview Scheduling: To schedule interviews and maintain interview records.
5. HR Analytics: To analyze recruitment data and generate statistical summaries.
6. Candidate Skill Matching: To compare candidate skills with the required job skills.
7. Recruitment Reporting: To display recruitment statistics and charts for better understanding.
8. Candidate Search: To quickly search and filter candidates based on available information.
9. Data Management: To store and retrieve recruitment records using a database.
10. Decision Support: To help HR personnel organize recruitment information and support human decision-making.
11. Data Export: To export candidate and recruitment records into CSV files for further analysis.
12. Recruitment Process Improvement: To reduce manual record-keeping and make recruitment activities more systematic.

CHAPTER 6

]

CONCLUSION

The Recruitment & HR Analytics Portal was successfully developed as a Python-based educational project to demonstrate the practical application of programming concepts in recruitment management and human resource analytics. The main objective of the project was to understand how technology can simplify recruitment-related activities, organize candidate information, and support data-driven analysis. The application provides a structured platform for managing candidate records and monitoring different stages of the recruitment process.

In conclusion, the Recruitment & HR Analytics Portal provides a useful foundation for understanding how Python programming and database technologies can be applied to human resource management. The project successfully demonstrates the basic principles of recruitment automation, candidate tracking, and HR data analysis in an educational setting. It also creates opportunities for further learning and development in web application development, database systems, data analytics, and artificial intelligence. Overall, this experiential learning project strengthened practical programming skills and provided valuable insight into the role of technology in making recruitment processes more organized, efficient, and data-oriented.

REFERENCES

[1] Python Software Foundation, *Python 3 Documentation*. https://docs.python.org/3/
[2] Python Software Foundation, *CSV File Reading and Writing*. https://docs.python.org/3/library/csv.html
[3] Python Software Foundation, *Collections — Container Datatypes*. https://docs.python.org/3/library/collections.html

[4] Python Software Foundation, *Datetime — Basic Date and Time Types*. https://docs.python.org/3/library/dateti me.html